

Comparative Analysis: Retail Sales Data Analysis Using Python

Code Alpha Internship Task :-3 Data visualizations

Project Overview

This project focuses on analyzing retail sales data using Python to identify important business trends, customer behavior, product performance, and profitability patterns.

The analysis uses Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn to transform raw sales data into meaningful visual insights.

Objectives

- Analyze overall sales and profit performance
- Identify the best-performing categories and sub-categories
- Analyze regional and state-wise sales
- Understand customer segment performance
- Analyze sales and profit trends over time
- Identify top and bottom-performing products
- Understand the impact of discounts on profitability
- Generate actionable business insights

1. Import Libraries

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Load Datasets

```
In [24]: df = pd.read_csv("Sample_ Superstore.csv")

df.head()
```

Out[24]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Segment	Country	City
0	1	CA-2016-152156	11-08-2016	11-11-2016	Second Class	CG-12520	Consumer	United States	Henderson
1	2	CA-2016-152156	11-08-2016	11-11-2016	Second Class	CG-12520	Consumer	United States	Henderson
2	3	CA-2016-138688	06-12-2016	6/16/2016	Second Class	DV-13045	Corporate	United States	Los Angeles
3	4	US-2015-108966	10-11-2015	10/18/2015	Standard Class	SO-20335	Consumer	United States	Fort Lauderdale
4	5	US-2015-108966	10-11-2015	10/18/2015	Standard Class	SO-20335	Consumer	United States	Fort Lauderdale

In [25]: `df.tail()`

Out[25]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Segment	Country	
9989	9990	CA-2014-110422	1/21/2014	1/23/2014	Second Class	TB-21400	Consumer	United States	
9990	9991	CA-2017-121258	2/26/2017	03-03-2017	Standard Class	DB-13060	Consumer	United States	Co
9991	9992	CA-2017-121258	2/26/2017	03-03-2017	Standard Class	DB-13060	Consumer	United States	Co
9992	9993	CA-2017-121258	2/26/2017	03-03-2017	Standard Class	DB-13060	Consumer	United States	Co
9993	9994	CA-2017-119914	05-04-2017	05-09-2017	Second Class	CC-12220	Consumer	United States	We

◀──▶

```
In [26]: df.shape
```

Out[26]: (9994, 19)

```
In [27]: df.columns
```

Out[27]: Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Segment', 'Country', 'City', 'State', 'Region', 'Product ID', 'Category', 'Sub-Category', 'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'], dtype='object')

```
In [28]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Row ID                 9994 non-null  int64
1   Order ID               9994 non-null  object
2   Order Date             9994 non-null  object
3   Ship Date              9994 non-null  object
4   Ship Mode              9994 non-null  object
5   Customer ID            9994 non-null  object
6   Segment                9994 non-null  object
7   Country                9994 non-null  object
8   City                   9994 non-null  object
9   State                  9994 non-null  object
10  Region                 9994 non-null  object
11  Product ID             9994 non-null  object
12  Category               9994 non-null  object
13  Sub-Category           9994 non-null  object
14  Product Name           9994 non-null  object
15  Sales                   9994 non-null  float64
16  Quantity               9994 non-null  int64
17  Discount               9994 non-null  float64
18  Profit                 9994 non-null  float64
dtypes: float64(3), int64(2), object(14)
memory usage: 1.4+ MB

```

In [29]: `df.describe()`

Out[29]:

	Row ID	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	4997.500000	229.858001	3.789574	0.156203	28.656896
std	2885.163629	623.245101	2.225110	0.206452	234.260108
min	1.000000	0.444000	1.000000	0.000000	-6599.978000
25%	2499.250000	17.280000	2.000000	0.000000	1.728750
50%	4997.500000	54.490000	3.000000	0.200000	8.666500
75%	7495.750000	209.940000	5.000000	0.200000	29.364000
max	9994.000000	22638.480000	14.000000	0.800000	8399.976000

3.Missing values

In [30]: `df.isnull().sum()`

```
Out[30]: Row ID      0
Order ID      0
Order Date    0
Ship Date     0
Ship Mode     0
Customer ID   0
Segment       0
Country       0
City          0
State         0
Region        0
Product ID    0
Category      0
Sub-Category  0
Product Name  0
Sales         0
Quantity      0
Discount      0
Profit        0
dtype: int64
```

4. Duplicate Records

```
In [31]: df.duplicated().sum()
```

```
Out[31]: np.int64(0)
```

```
In [59]: # Convert Order Date correctly
df["Order Date"] = pd.to_datetime(
    df["Order Date"],
    format="mixed"
)

# Create time-related columns
df["Year"] = df["Order Date"].dt.year
df["Month"] = df["Order Date"].dt.month
df["Month_Name"] = df["Order Date"].dt.month_name()
df["Year-Month"] = df["Order Date"].dt.to_period("M")

# Check the result
df[["Order Date", "Year", "Month", "Month_Name", "Year-Month"]].head()
```

```
Out[59]:
```

	Order Date	Year	Month	Month_Name	Year-Month
0	2016-11-08	2016	11	November	2016-11
1	2016-11-08	2016	11	November	2016-11
2	2016-06-12	2016	6	June	2016-06
3	2015-10-11	2015	10	October	2015-10
4	2015-10-11	2015	10	October	2015-10

5. Key Business KPIs

```
In [34]: total_sales = df["Sales"].sum()
total_profit = df["Profit"].sum()
total_quantity = df["Quantity"].sum()
```

```
total_orders = df["Order ID"].nunique()

print("Total Sales:", round(total_sales, 2))
print("Total Profit:", round(total_profit, 2))
print("Total Quantity:", total_quantity)
print("Total Orders:", total_orders)
```

Total Sales: 2297200.86

Total Profit: 286397.02

Total Quantity: 37873

Total Orders: 5009

```
In [35]: profit_margin = (total_profit / total_sales) * 100

print("Profit Margin:", round(profit_margin, 2), "%")
```

Profit Margin: 12.47 %

6.Sales by Category

```
In [36]: category_sales = (
    df.groupby("Category")["Sales"]
      .sum()
      .sort_values(ascending=False)
    )

category_sales
```

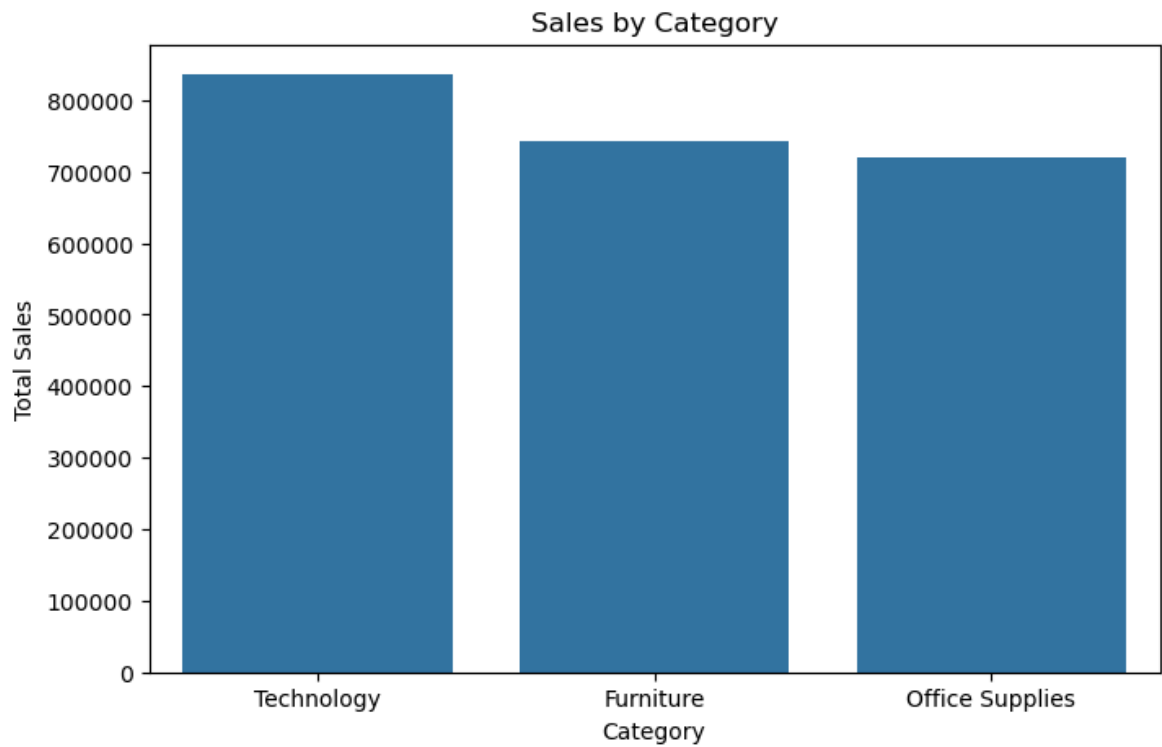
```
Out[36]: Category
Technology      836154.0330
Furniture       741999.7953
Office Supplies  719047.0320
Name: Sales, dtype: float64
```

7.Visualization

```
In [39]: plt.figure(figsize=(8,5))

sns.barplot(
    x=category_sales.index,
    y=category_sales.values
)

plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.show()
```

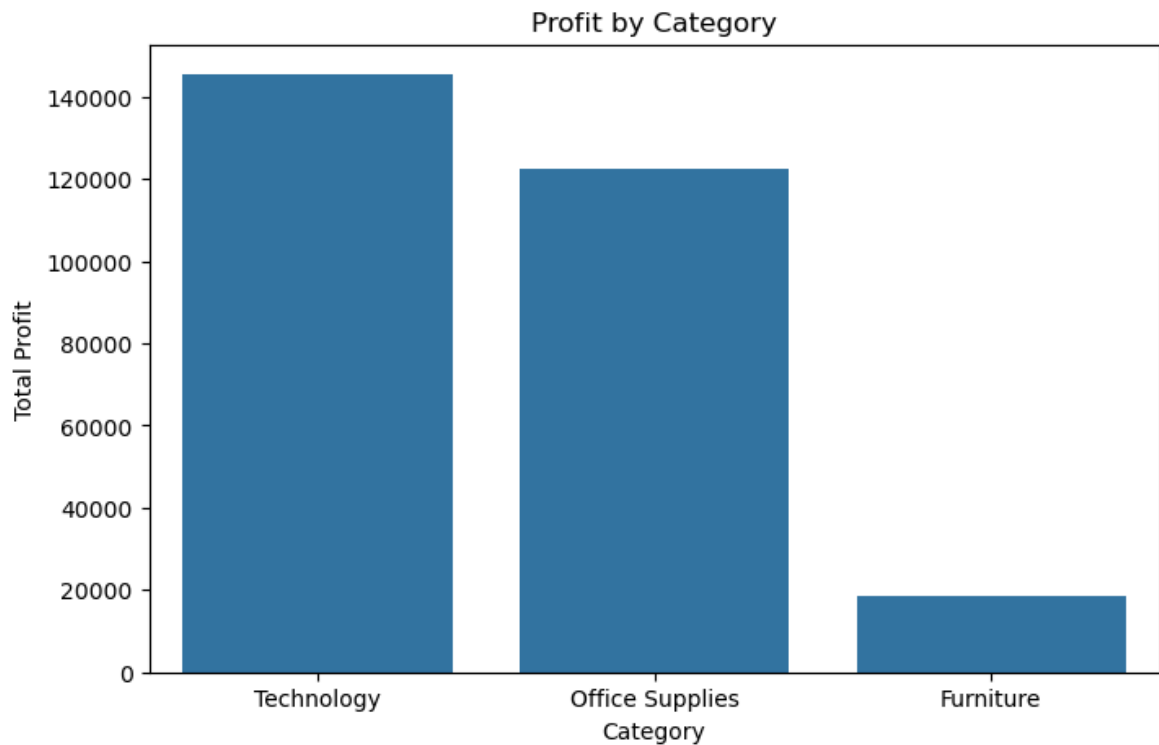


Business Insight

The Technology category generates the highest sales, followed by Furniture and Office Supplies. This indicates that Technology products are a major revenue contributor.

8. Profit by Category

```
In [40]: category_profit = (  
    df.groupby("Category")["Profit"]  
    .sum()  
    .sort_values(ascending=False)  
    )  
  
plt.figure(figsize=(8,5))  
  
sns.barplot(  
    x=category_profit.index,  
    y=category_profit.values  
    )  
  
plt.title("Profit by Category")  
plt.xlabel("Category")  
plt.ylabel("Total Profit")  
plt.show()
```



Business Insight

Technology is the most profitable category, generating approximately \$145K in total profit.

Office Supplies is also highly profitable, while Furniture contributes significantly less profit compared with the other categories.

This suggests that Furniture requires further investigation regarding pricing, discounts, product costs, and profitability.

9. Top 10 States by Sales

```
In [41]: top_states_sales = (
    df.groupby("State")["Sales"]
      .sum()
      .sort_values(ascending=False)
      .head(10)
    )

top_states_sales
```

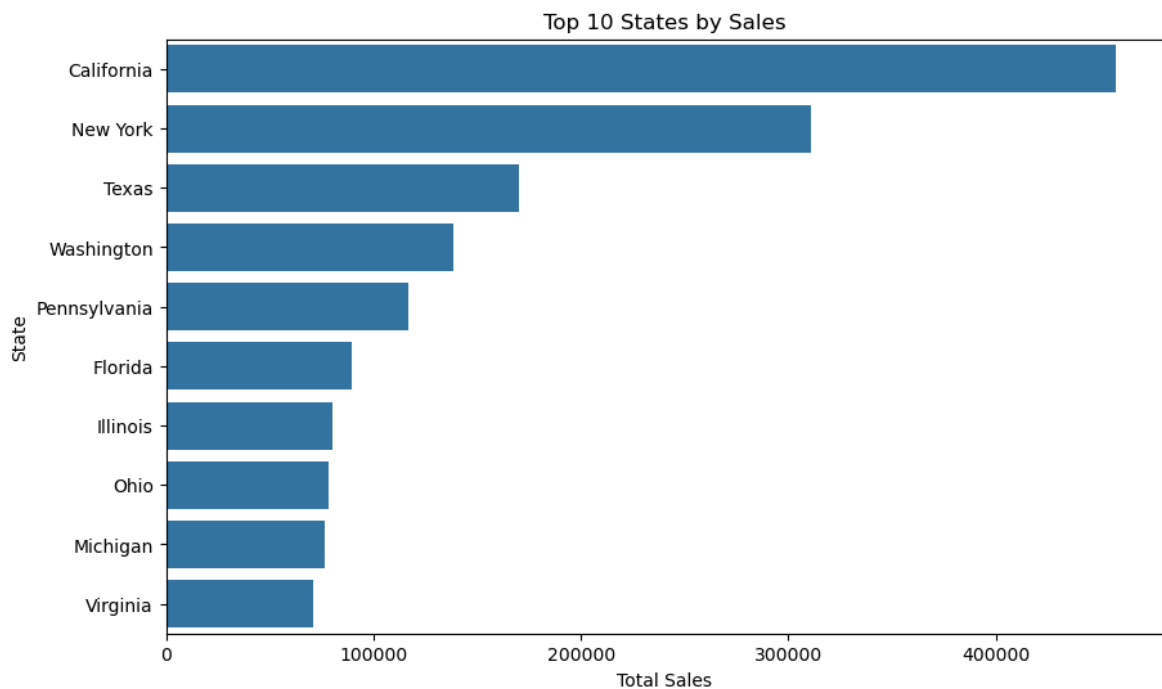
```
Out[41]: State
California    457687.6315
New York      310876.2710
Texas         170188.0458
Washington    138641.2700
Pennsylvania  116511.9140
Florida       89473.7080
Illinois      80166.1010
Ohio          78258.1360
Michigan      76269.6140
Virginia      70636.7200
Name: Sales, dtype: float64
```



```
In [42]: plt.figure(figsize=(10,6))

sns.barplot(
    x=top_states_sales.values,
    y=top_states_sales.index
)

plt.title("Top 10 States by Sales")
plt.xlabel("Total Sales")
plt.ylabel("State")
plt.show()
```



Business Insight

California is the leading state in terms of sales, followed by New York and Texas. These states represent important markets and should receive continued attention in sales and marketing strategies.

10. Top 10 States by Profit

```
In [43]: top_states_profit = (
    df.groupby("State")["Profit"]
    .sum()
    .sort_values(ascending=False)
    .head(10)
)

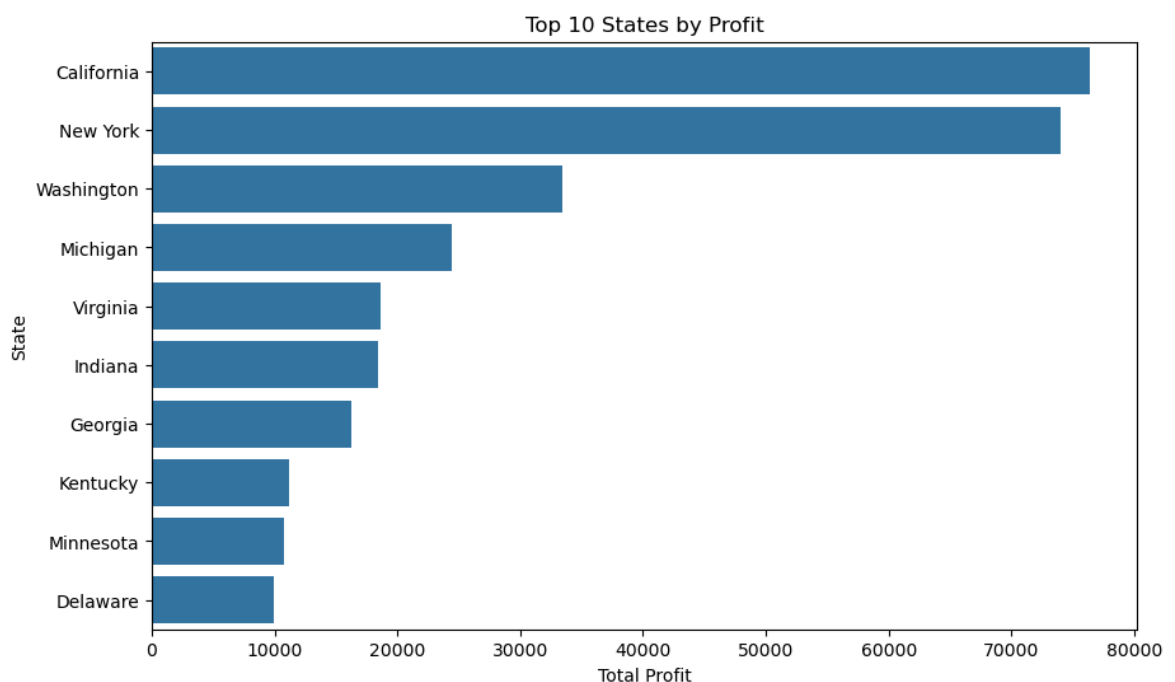
top_states_profit
```

```
Out[43]: State
California    76381.3871
New York      74038.5486
Washington    33402.6517
Michigan      24463.1876
Virginia      18597.9504
Indiana       18382.9363
Georgia       16250.0433
Kentucky      11199.6966
Minnesota     10823.1874
Delaware      9977.3748
Name: Profit, dtype: float64
```

```
In [44]: plt.figure(figsize=(10,6))

sns.barplot(
    x=top_states_profit.values,
    y=top_states_profit.index
)

plt.title("Top 10 States by Profit")
plt.xlabel("Total Profit")
plt.ylabel("State")
plt.show()
```



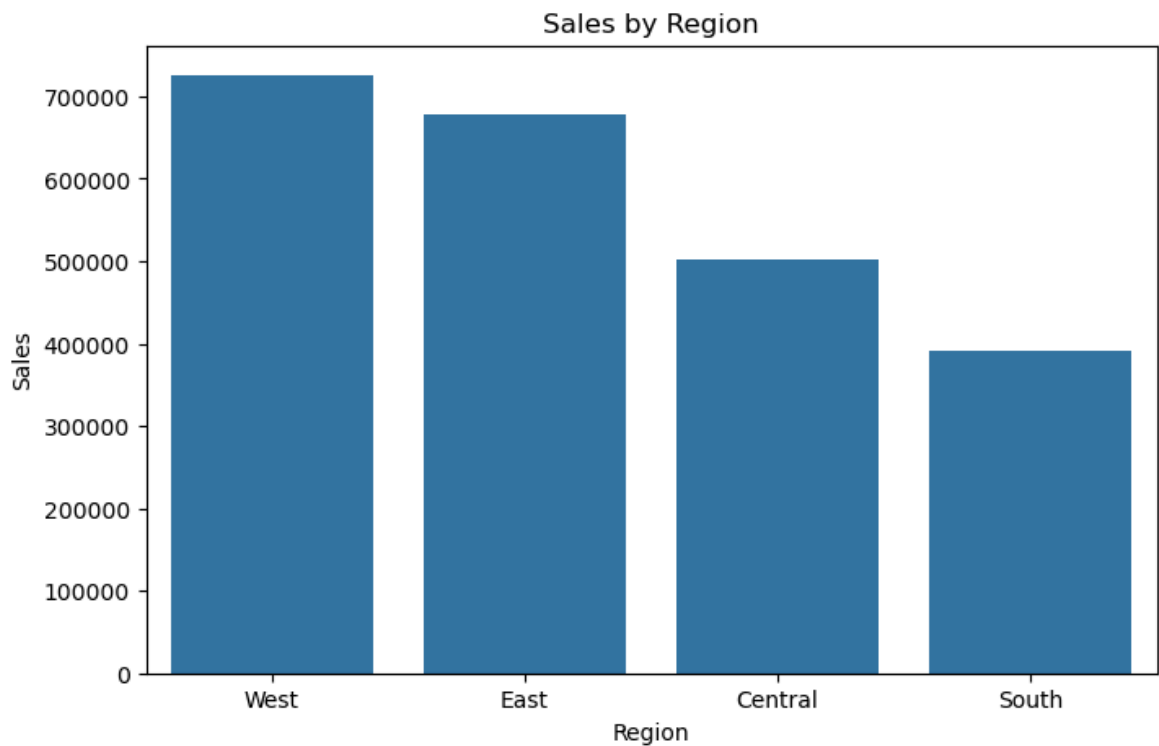
Business Insight

California and New York generate the highest profits. However, a state with high sales does not necessarily have proportionally high profit, demonstrating the importance of analyzing profitability alongside revenue.

11. Sales by Region

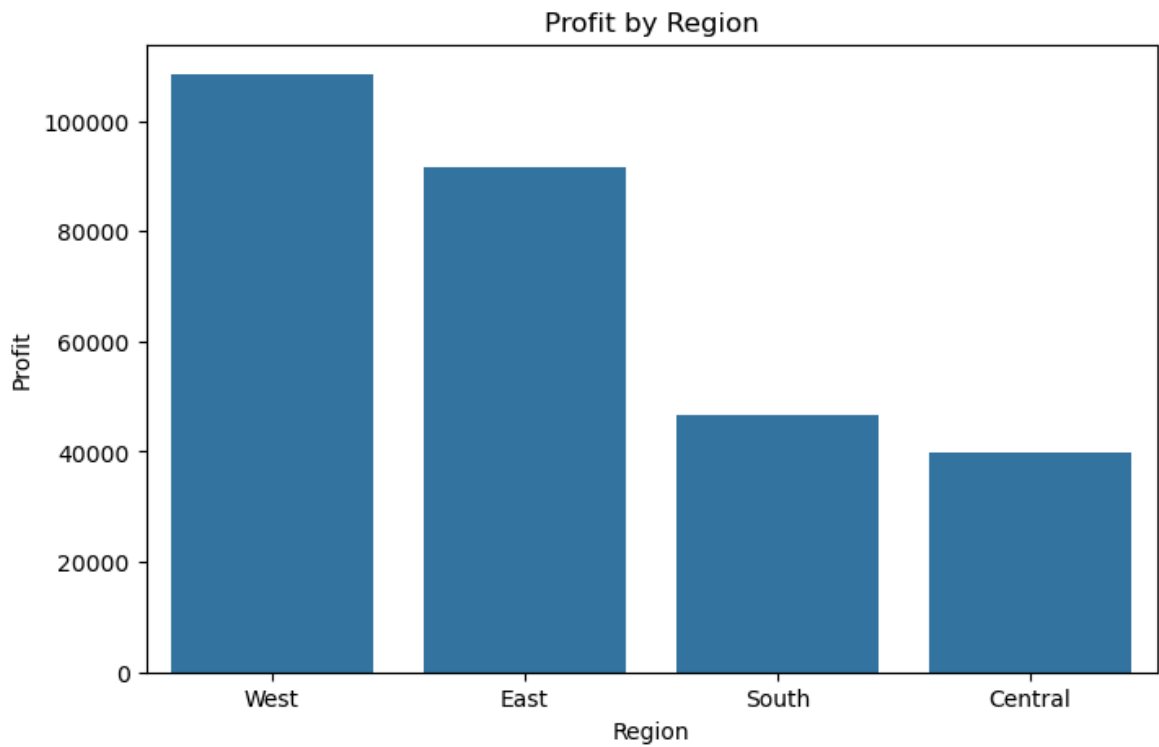
```
In [45]: region_sales = (
    df.groupby("Region")["Sales"]
    .sum()
    .sort_values(ascending=False)
```

```
)  
  
plt.figure(figsize=(8,5))  
  
sns.barplot(  
    x=region_sales.index,  
    y=region_sales.values  
)  
  
plt.title("Sales by Region")  
plt.xlabel("Region")  
plt.ylabel("Sales")  
plt.show()
```



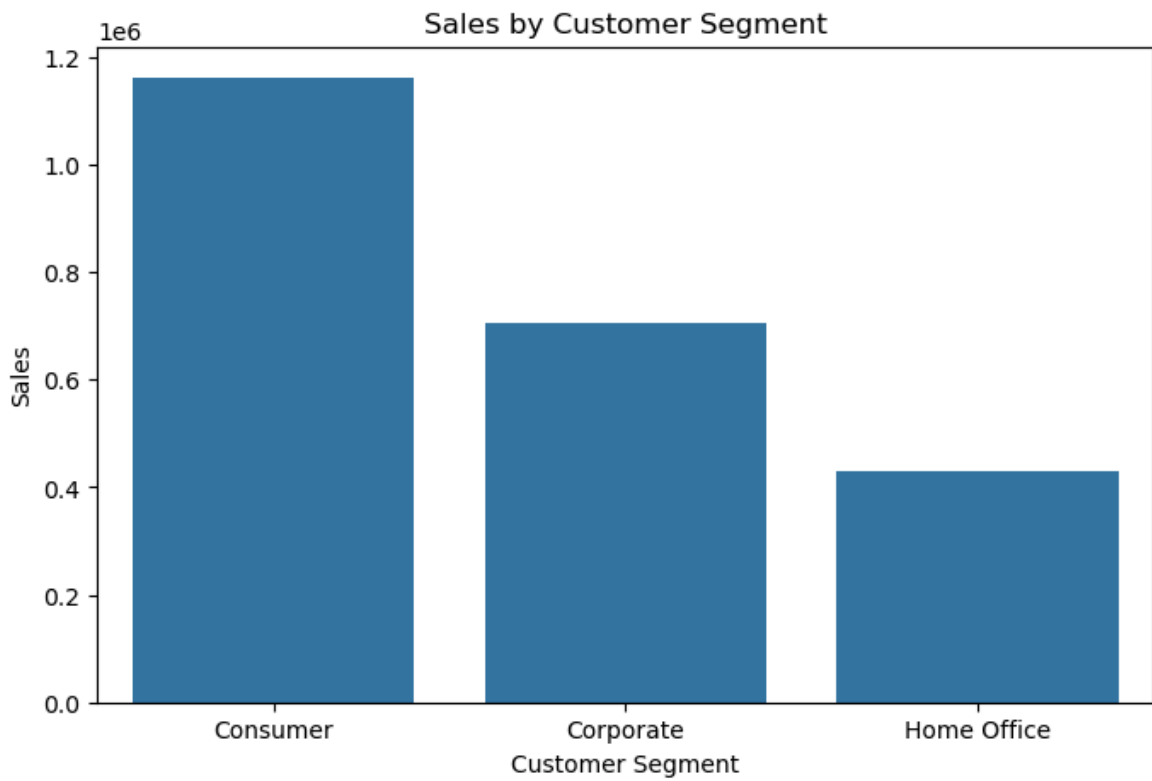
12. Profit by Region

```
In [46]: region_profit = (  
    df.groupby("Region")["Profit"]  
        .sum()  
        .sort_values(ascending=False)  
    )  
  
plt.figure(figsize=(8,5))  
  
sns.barplot(  
    x=region_profit.index,  
    y=region_profit.values  
)  
  
plt.title("Profit by Region")  
plt.xlabel("Region")  
plt.ylabel("Profit")  
plt.show()
```



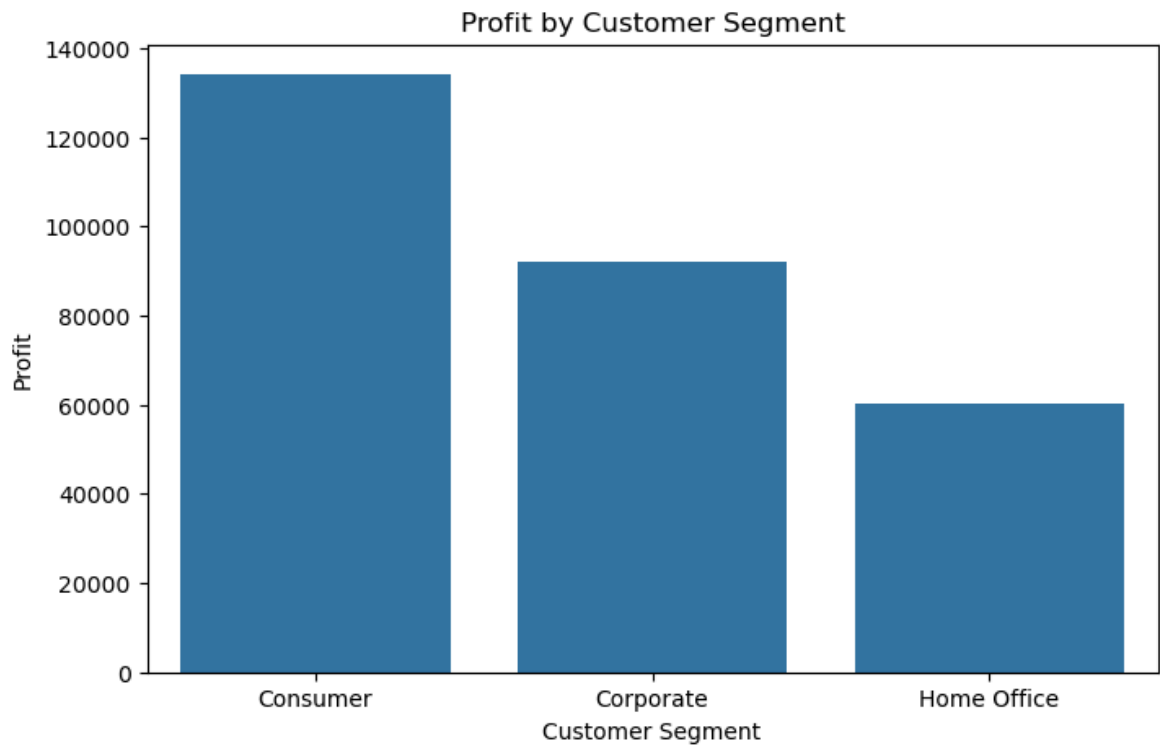
13. Sales by Customer Segment

```
In [47]: segment_sales = (  
    df.groupby("Segment")["Sales"]  
    .sum()  
    .sort_values(ascending=False)  
    )  
  
plt.figure(figsize=(8,5))  
  
sns.barplot(  
    x=segment_sales.index,  
    y=segment_sales.values  
    )  
  
plt.title("Sales by Customer Segment")  
plt.xlabel("Customer Segment")  
plt.ylabel("Sales")  
plt.show()
```



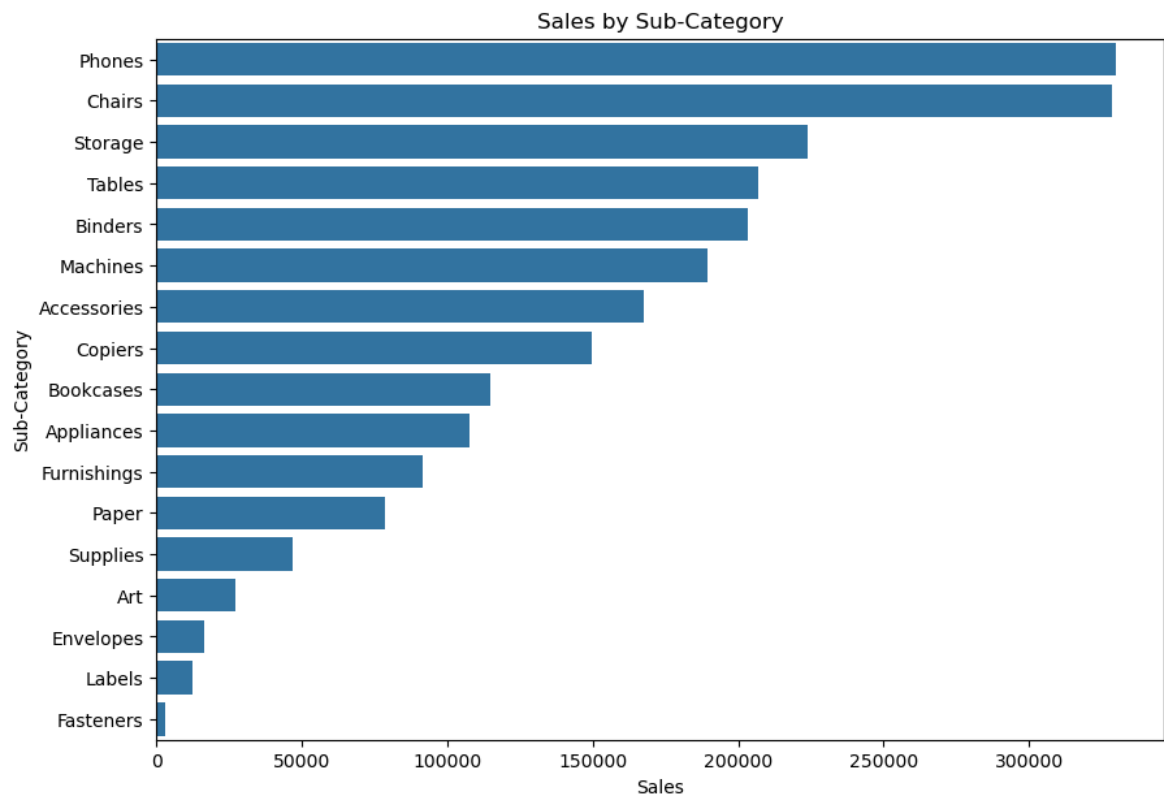
14. Profit by Customer Segment

```
In [50]: segment_profit = (  
    df.groupby("Segment")["Profit"]  
    .sum()  
    .sort_values(ascending=False)  
)  
  
plt.figure(figsize=(8,5))  
  
sns.barplot(  
    x=segment_profit.index,  
    y=segment_profit.values  
)  
  
plt.title("Profit by Customer Segment")  
plt.xlabel("Customer Segment")  
plt.ylabel("Profit")  
plt.show()
```



15. Sales by Sub-Category

```
In [51]: subcategory_sales = (  
    df.groupby("Sub-Category")["Sales"]  
        .sum()  
        .sort_values(ascending=False)  
    )  
  
plt.figure(figsize=(10,7))  
  
sns.barplot(  
    x=subcategory_sales.values,  
    y=subcategory_sales.index  
)  
  
plt.title("Sales by Sub-Category")  
plt.xlabel("Sales")  
plt.ylabel("Sub-Category")  
plt.show()
```



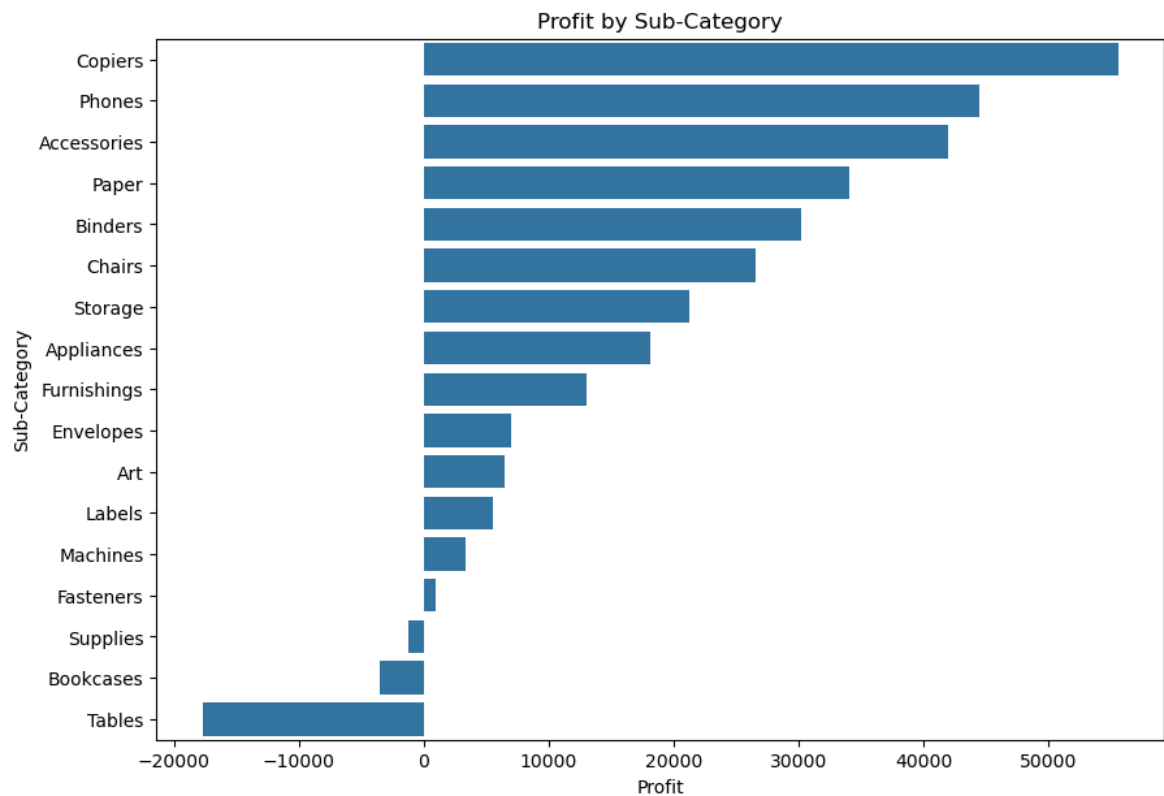
16. Profit by Sub-Category

```
In [52]: subcategory_profit = (
    df.groupby("Sub-Category")["Profit"]
    .sum()
    .sort_values(ascending=False)
)

plt.figure(figsize=(10,7))

sns.barplot(
    x=subcategory_profit.values,
    y=subcategory_profit.index
)

plt.title("Profit by Sub-Category")
plt.xlabel("Profit")
plt.ylabel("Sub-Category")
plt.show()
```



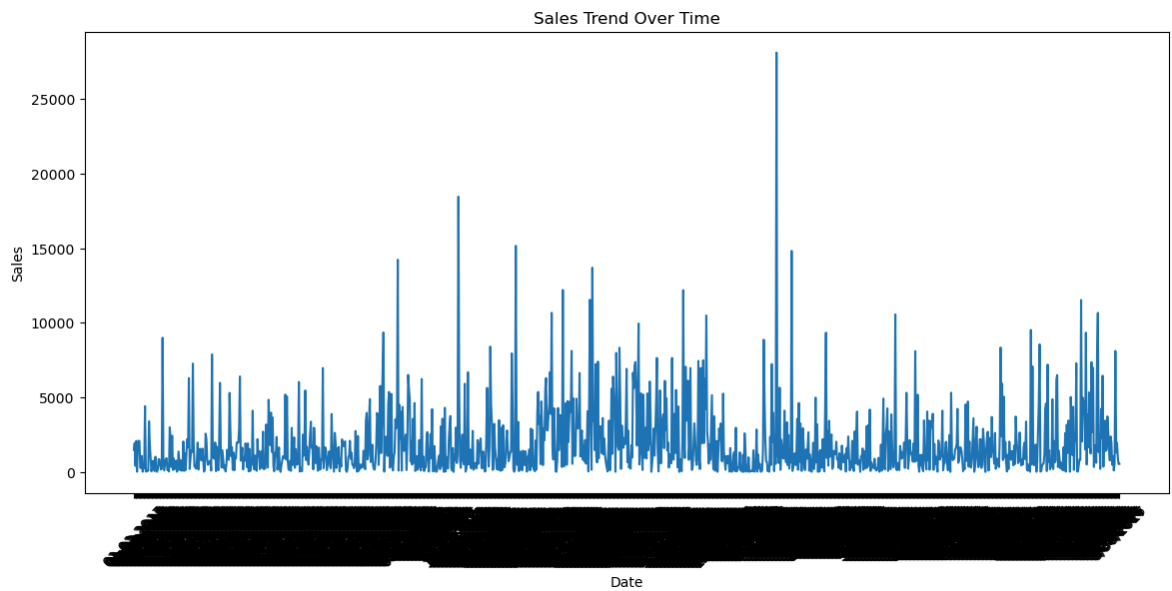
17. Monthly Sales Trend

```
In [55]: monthly_sales = (
    df.groupby("Order Date")["Sales"]
    .sum()
)

plt.figure(figsize=(14,6))

plt.plot(
    monthly_sales.index,
    monthly_sales.values
)

plt.title("Sales Trend Over Time")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.xticks(rotation=45)
plt.show()
```

18. Yearly Sales

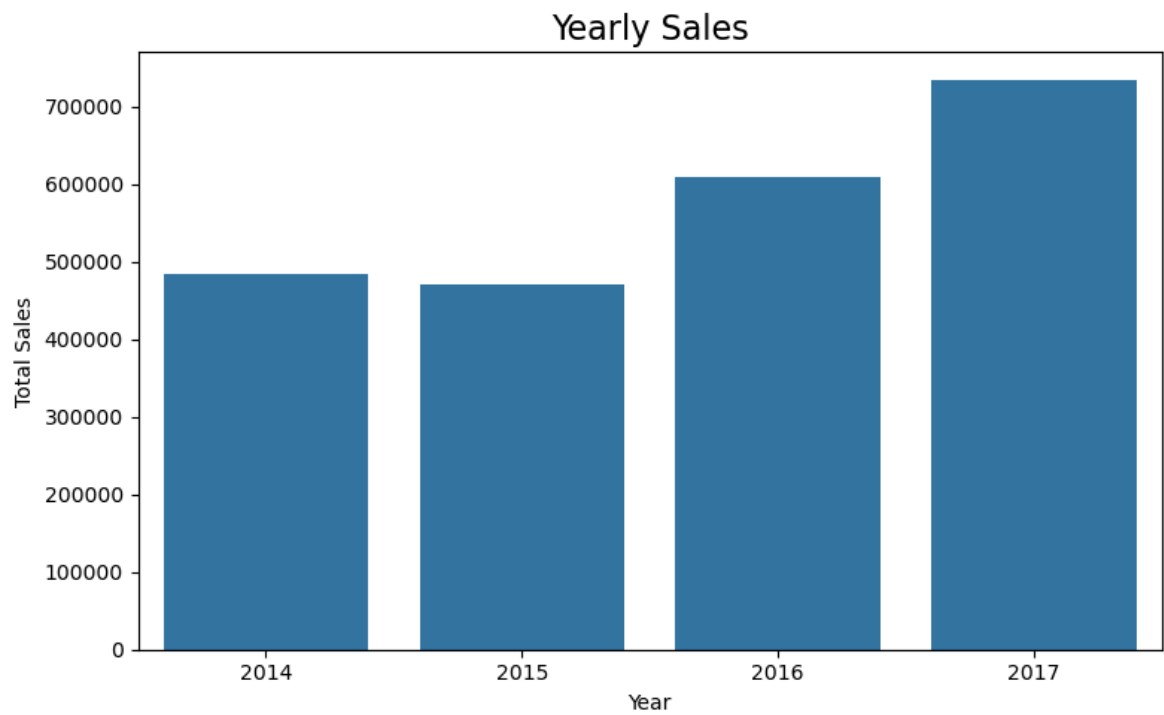
```
In [60]: yearly_sales = (
    df.groupby("Year")["Sales"]
      .sum()
      .sort_index()
    )

plt.figure(figsize=(8, 5))

sns.barplot(
    x=yearly_sales.index.astype(str),
    y=yearly_sales.values
)

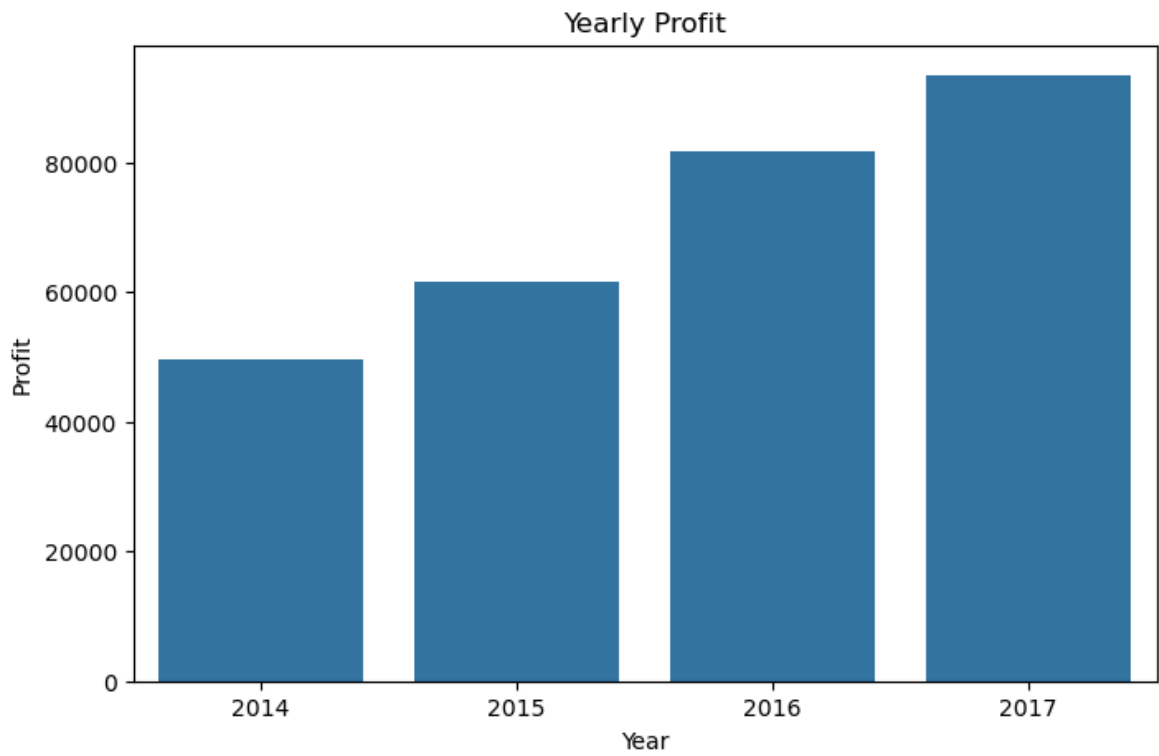
plt.title("Yearly Sales", fontsize=16)
plt.xlabel("Year")
plt.ylabel("Total Sales")

plt.tight_layout()
plt.show()
```



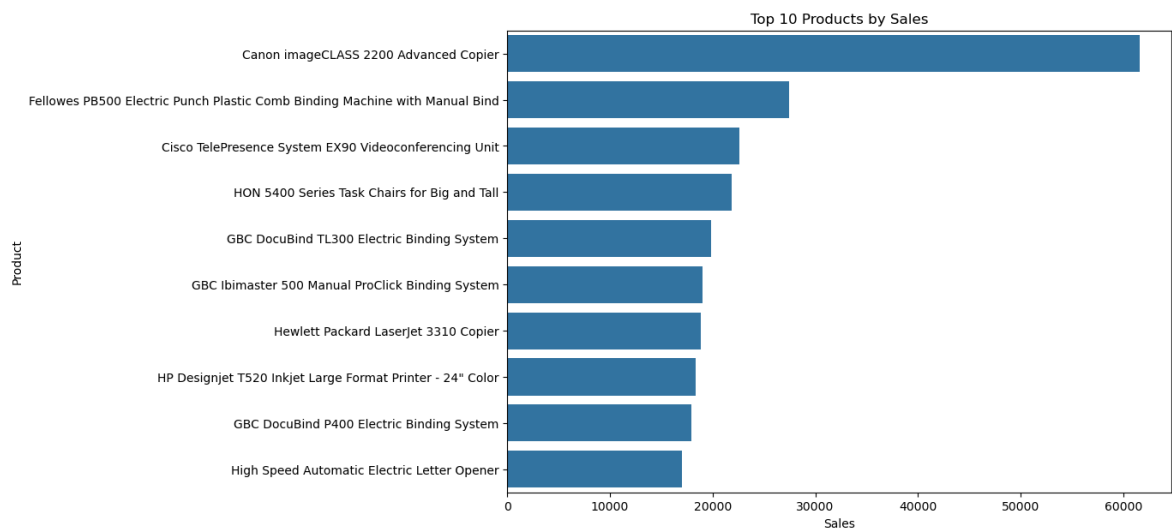
19. Yearly Profit

```
In [61]: yearly_profit = (  
    df.groupby("Year")["Profit"]  
    .sum()  
    )  
  
plt.figure(figsize=(8,5))  
  
sns.barplot(  
    x=yearly_profit.index.astype(str),  
    y=yearly_profit.values  
    )  
  
plt.title("Yearly Profit")  
plt.xlabel("Year")  
plt.ylabel("Profit")  
plt.show()
```



20. Top 10 Products

```
In [63]: top_products = (  
    df.groupby("Product Name")["Sales"]  
    .sum()  
    .sort_values(ascending=False)  
    .head(10)  
)  
  
plt.figure(figsize=(10,7))  
  
sns.barplot(  
    x=top_products.values,  
    y=top_products.index  
)  
  
plt.title("Top 10 Products by Sales")  
plt.xlabel("Sales")  
plt.ylabel("Product")  
plt.show()
```

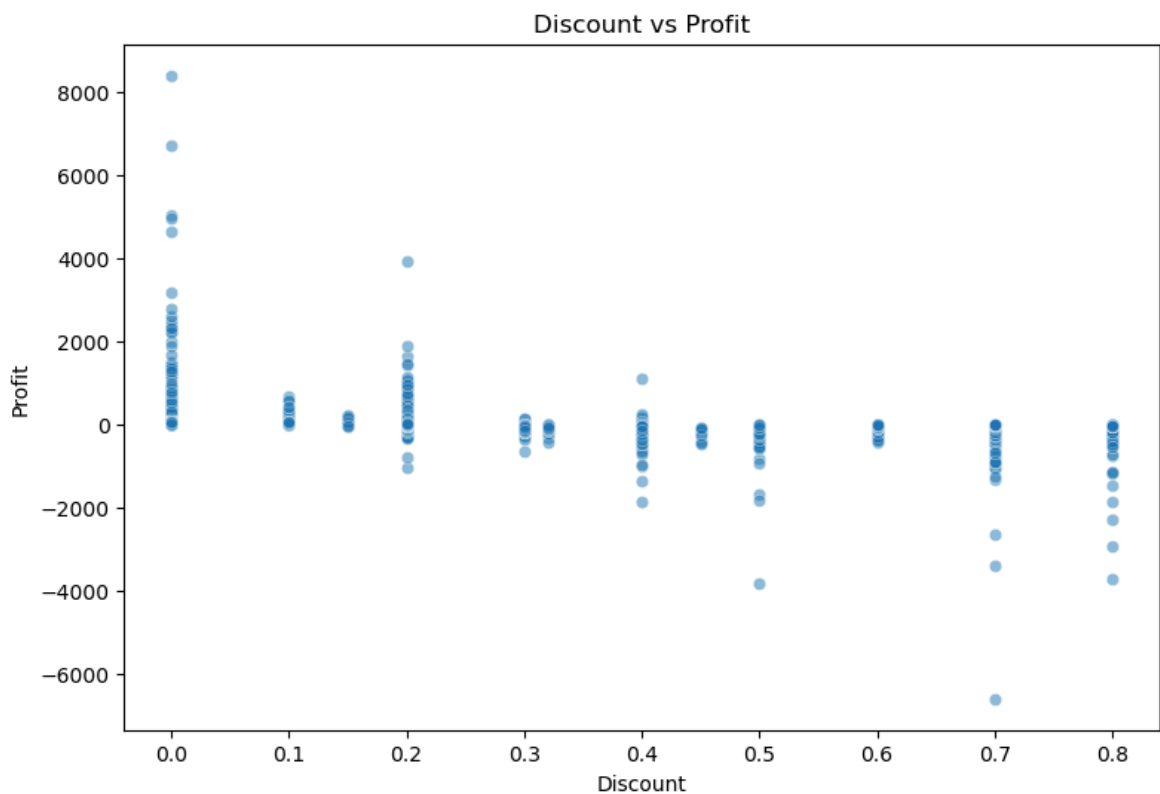


21. Discount vs Profit

```
In [64]: plt.figure(figsize=(9,6))

sns.scatterplot(
    data=df,
    x="Discount",
    y="Profit",
    alpha=0.5
)

plt.title("Discount vs Profit")
plt.xlabel("Discount")
plt.ylabel("Profit")
plt.show()
```



Business Insight

The relationship between discount and profit should be examined carefully. Higher discounts can reduce profitability, particularly for products that already have low margins.

The business should therefore identify discount levels that maximize sales without significantly reducing profit.

22. Sales vs Profit

```
In [65]: plt.figure(figsize=(9,6))

sns.scatterplot(
    data=df,
    x="Sales",
    y="Profit",
    hue="Category"
)

plt.title("Sales vs Profit by Category")
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.show()
```

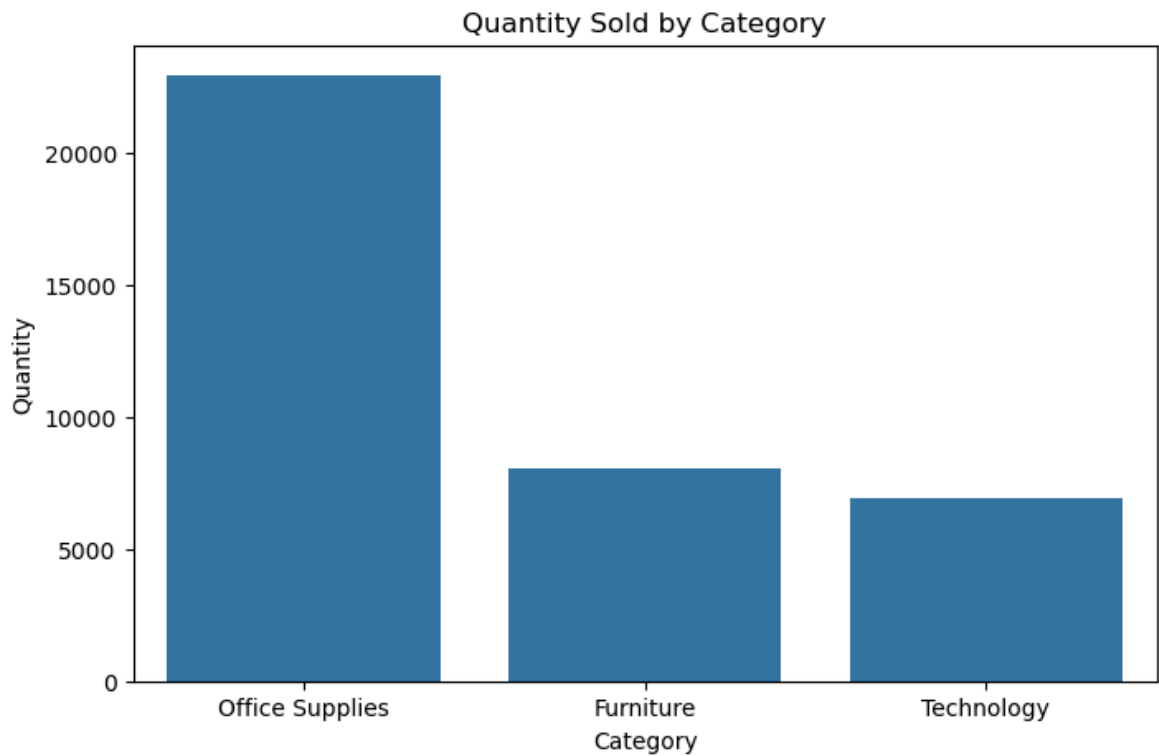


23. Quantity by Category

```
In [66]: quantity_category = (
    df.groupby("Category")["Quantity"]
    .sum()
    .sort_values(ascending=False)
)

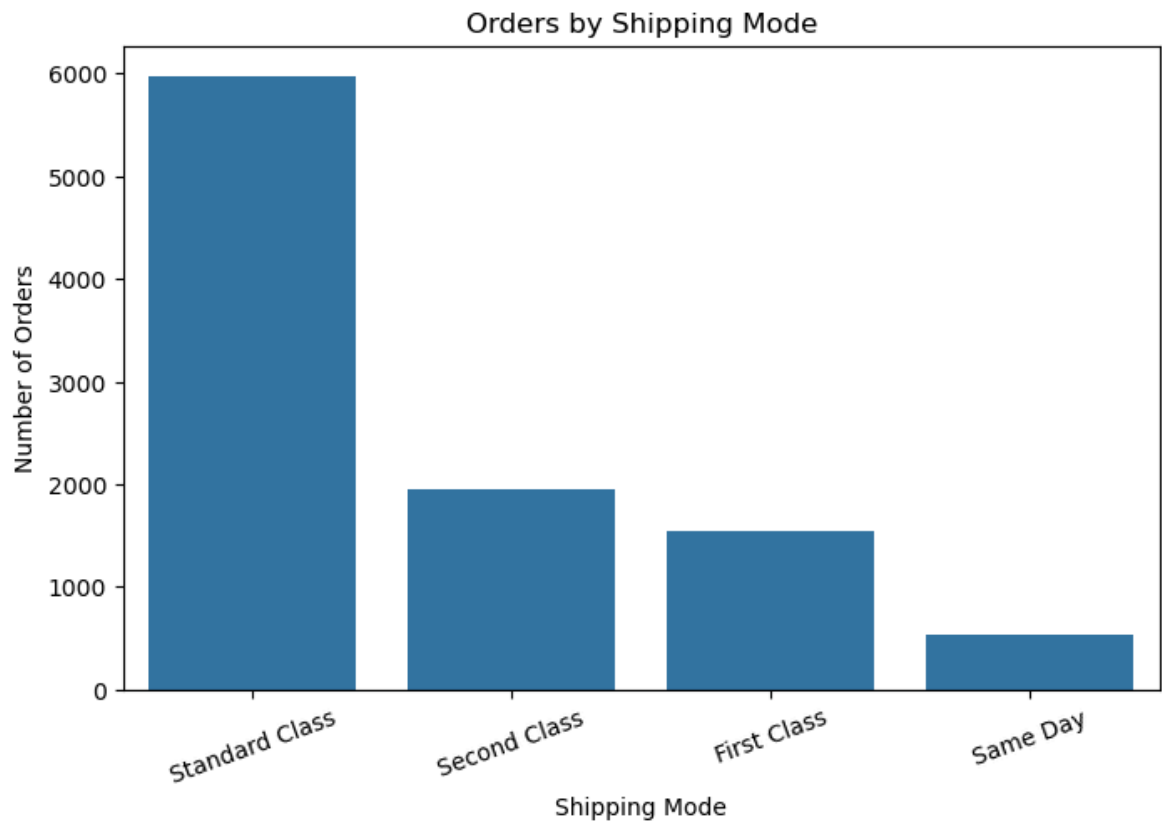
plt.figure(figsize=(8,5))
```

```
sns.barplot(  
    x=quantity_category.index,  
    y=quantity_category.values  
)  
  
plt.title("Quantity Sold by Category")  
plt.xlabel("Category")  
plt.ylabel("Quantity")  
plt.show()
```



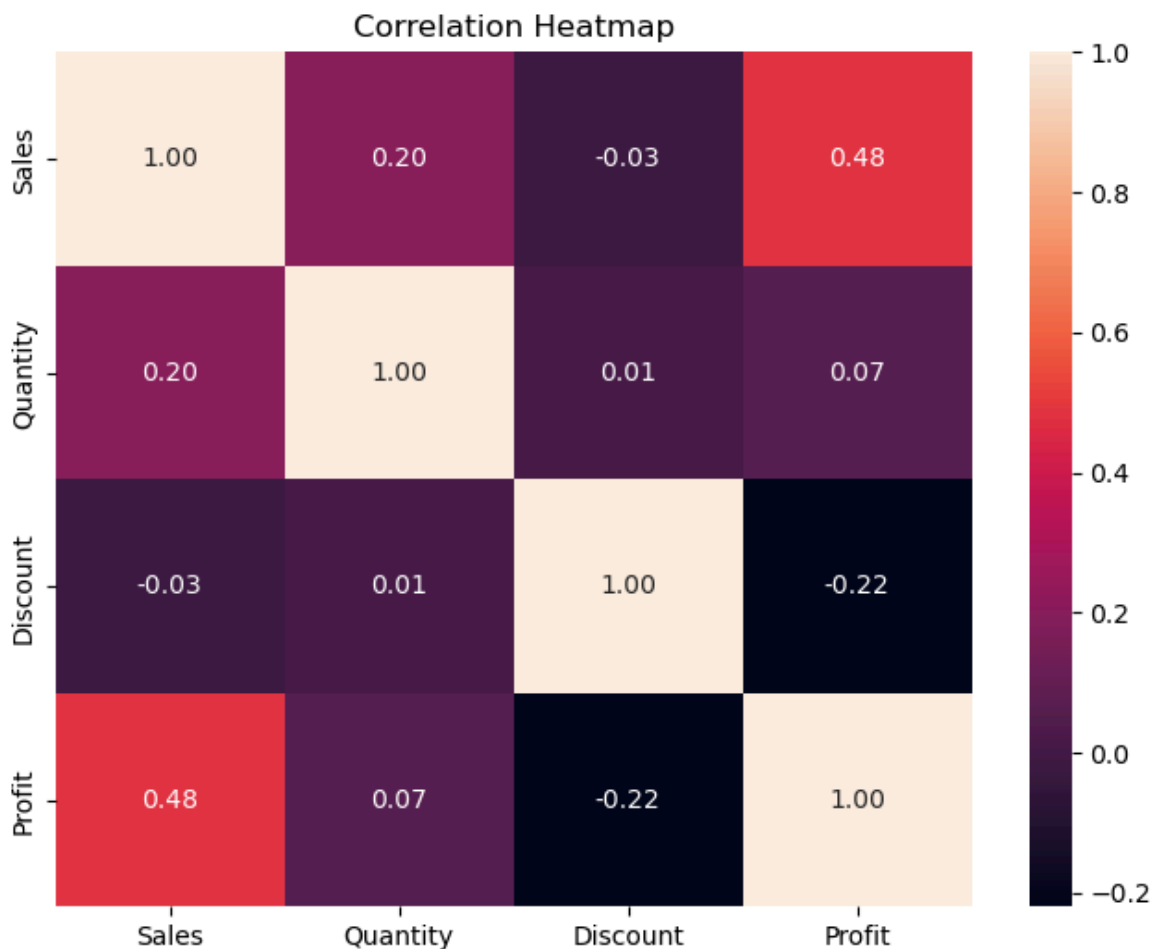
24. Shipping Mode Analysis

```
In [67]: shipping = df["Ship Mode"].value_counts()  
  
plt.figure(figsize=(8,5))  
  
sns.barplot(  
    x=shipping.index,  
    y=shipping.values  
)  
  
plt.title("Orders by Shipping Mode")  
plt.xlabel("Shipping Mode")  
plt.ylabel("Number of Orders")  
plt.xticks(rotation=20)  
plt.show()
```



25. Correlation Heatmap

```
In [68]: numeric_columns = [  
    "Sales",  
    "Quantity",  
    "Discount",  
    "Profit"  
]  
  
correlation = df[numeric_columns].corr()  
  
plt.figure(figsize=(8,6))  
  
sns.heatmap(  
    correlation,  
    annot=True,  
    fmt=".2f"  
)  
  
plt.title("Correlation Heatmap")  
plt.show()
```



Final Business Insights

1. Category Performance

Technology is the strongest category in terms of both sales and profit, making it an important contributor to overall business performance.

2. State Performance

California and New York are among the strongest states in terms of sales and profitability.

3. Regional Performance

Sales and profit vary across regions, indicating opportunities for region-specific strategies.

4. Customer Segments

Different customer segments contribute differently to total sales and profit. Understanding these segments can help improve marketing strategies.

5. Product Performance

A small number of products contribute significantly to total sales. These products should receive appropriate inventory and marketing attention.

6. Discount Impact

Discounting should be carefully controlled because excessive discounts can reduce profitability.

7. Sales and Profit

High sales do not always guarantee high profit. Therefore, business decisions should consider both revenue and profitability.

8. Overall Recommendation

The business should focus on high-profit products and regions, optimize discount strategies, maintain inventory for high-performing products, and develop targeted strategies for weaker categories and markets.